

AWS ParallelCluster Setup & Testing Guide

By Andrew Ooi Yu Zhang, School of Information Technology, Monash University Malaysia

Purpose

This guide documents the setup and testing process used to evaluate AWS ParallelCluster as a potential AWS-based alternative to the CAAS HPC environment. It focuses on a Slurm cluster for CPU workloads and includes the troubleshooting steps encountered during setup.

Part A - Tool Installation

1. Architecture

The intended architecture is:

AWS account → VPC/subnet → ParallelCluster head node → Slurm scheduler → EC2 compute nodes

2. Prerequisites

- ☐ An AWS account with permissions to create/manage ParallelCluster resources
- ☐ AWS CLI installed and configured
- ☐ AWS ParallelCluster CLI installed
- ☐ An EC2 key pair for SSH access
- ☐ A VPC and subnet suitable for the cluster
- ☐ A region selected for evaluation. (e.g. ap-southeast-2), but any region should work
- ☐ A billing method

3. Install Python

AWS ParallelCluster's current pip installation requires Python 3.9 or later.

1. Download Python for your operating system from: <https://www.python.org/downloads/>
2. Verify using the command `python3 --version`
3. On Windows, if `python3` is not recognized, use `py --version` instead.

4. Install AWS CLI

1. Download and install AWS CLI for your operating system using the official AWS instructions: <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>
 - a. For MacOS, please note that the installer will install to `.local/share`, therefore I recommend the GUI installer instead which installs to `local/bin`, which should automatically add it to `PATH`.
2. After installation, close and reopen your terminal/PowerShell window.
3. Verify using the command `aws --version`
 - a. You should receive an AWS CLI version string.

- b. If `aws` is not found, the CLI is either not installed correctly or its installation directory is not in PATH.

5. Install Node.js

AWS ParallelCluster uses AWS CDK for CloudFormation template generation, which requires Node.js.

1. Install a supported LTS version from: <https://nodejs.org/en/download>
2. Verify using `node --version` or `npm --version`

6. Install AWS ParallelCluster

AWS recommends installing ParallelCluster in a Python virtual environment.

Here's the official ParallelCluster installation documentation:

<https://docs.aws.amazon.com/parallelcluster/latest/ug/install-v3-virtual-environment.html>

On Windows:

```
py -m venv $HOME\parallelcluster-env & $HOME\parallelcluster-env\Scripts\Activate.ps1
py -m pip install --upgrade pip
py -m pip install --upgrade aws-parallelcluster
```

On macOS/Linux:

```
python3 -m venv ~/parallelcluster-env
source ~/parallelcluster-env/bin/activate
python3 -m pip install --upgrade pip
python3 -m pip install --upgrade aws-parallelcluster
```

Verify:

```
pcluster version
```

Part B - Configure AWS

7. Open AWS

Go to [AWS Management Console](#). If you do not already have an AWS account, create one and complete the account registration process. You will be required to also provide a payment method, but please do note that as long as you are on the free plan, you won't be charged. A new account also gets 100 USD worth of free credits to start, with the option to get 300 - 400 USD more free credits by doing some simple tasks.

8. Create IAM user for ParallelCluster

From the AWS Management Console:

1. Search for **IAM** in the search bar at the top.
2. Navigate to **Iam users** and select **Create user**.
3. Put any name and click **Next**
4. In permissions, select **Attach policies directly**.
5. Select **AdministratorAccess** and click **Next**.
6. Click **Create user**.

9. Create an access key

1. Navigate to your newly created user in **Iam users** and click on the user.
2. Select **Security credentials** and scroll down to **Access keys**, click on **Create access key**.
3. Select **Command Line Interface (CLI)**, confirm and create.
4. Now copy down the access key ID and the secret access key.
 - a. You won't be able to see the secret access key again, so please keep it securely.
 - b. If you do lose it, you can just recreate another one.

10. Configure AWS credentials

1. Configure the AWS identity that will administer the cluster: `aws configure`
2. The AWS CLI will prompt you with:
 - a. AWS Access Key ID [None]:
 - b. AWS Secret Access Key [None]:
 - c. Default region name [None]:
 - d. Default output format [None]:
3. Enter your access key ID and secret access key from earlier.
4. For the region name, you can put `ap-southeast-2`, which is Asia Pacific (Sydney).
 - a. Please feel free to put any other regions, you can refer to all possible regions [here](#).
5. For the output format, you can put `json`.

11. Verify AWS authentication

1. Run `aws sts get-caller-identity`, you should see your user ID, account ID and Arn.
2. Run `aws configure get region`, you should see your selected region.
3. Run `aws ec2 describe-availability-zones`, you should see Malaysia availability zones, this verifies that the CLI is successfully making EC2 API calls.
4. Run `aws cloudformation list-stacks --max-items 1`, this verifies that the CLI can call CloudFormation which is required for cluster operations.
5. Press **CTRL+C** to exit out of any of the commands above that are too long.

12. Creating ParallelCluster SSH Key Pair

1. Confirm that your console is in the region you selected, in our example case it should be `ap-southeast-2`.
2. Search **key pairs** in the search bar on the top and navigate there.
3. Click on **Create key pair**.
4. Give a name for the key pair, **RSA** as key pair type and **.pem** as private key file format.
5. Click on **Create key pair**, the private key will be downloaded to your computer.
6. Create a working folder and move the private key file into it.
7. In your terminal, set the private key's permissions using `chmod 400 <KEY-NAME>.pem`
8. Verify using `aws ec2 describe-key-pairs --key-names <KEY-NAME>`

13. Configuring the ParallelCluster

1. Download `cluster.yaml` [here](#)
2. In the same working folder, place the file `cluster.yaml`.
3. In the AWS Console, search for **VPC** and navigate to **Your VPCs**.
4. Find the ID for your default VPC.
 - a. You can also check via CLI using this command (remember to change the region to your own):
`aws ec2 describe-vpcs --filters "Name=is-default,Values=true" --region ap-southeast-2`
5. Now go to **Subnets**, copy the subnet ID of one of your subnets with the corresponding VPC ID.

6. Replace **region**, **subnet ID** and **key name** in the file.
7. Validate the configuration using this command (you can set the cluster name to anything you'd like, and remember to change the region to your own): `pcluster create-cluster --cluster-name mycluster --cluster-configuration cluster.yaml --region ap-southeast-2 --dryrun true`
8. If it says the dry run would've succeeded, proceed to run without the `--dryrun true` flag.
9. This process will take a few minutes, you can monitor the progress using this command: `pcluster describe-cluster --cluster-name mycluster --region ap-southeast-2`
10. Once it says **CREATE_COMPLETE**, we can SSH into the cluster.

14. Accessing the ParallelCluster

1. SSH into the cluster using this command: `pcluster ssh --cluster-name mycluster --region ap-southeast-2 -i <KEY_NAME>.pem`, enter **yes** when prompted.
 - a. If that doesn't work, get the head-node IP using `pcluster describe-cluster --cluster-name mycluster --region ap-southeast-2`
 - b. Find **publicIpAddress** and do this instead: `ssh -i parallelcluster-key.pem ec2-user@<publicIpAddress>`
2. Once that's done, we are in the head node. Let's verify the slurm using `sinfo` and `squeue`

15. Submitting a Job

1. We will use [hello.c](#) as a sample file, use `nano` to create and edit files inside the slurm.
2. Compile the program using `gcc hello.c -o hello`
3. Create the Slurm batch file, [here's](#) a completed template.
4. Run `sbatch job.sh`, you should see the job being submitted.
5. Run `squeue` and `sinfo` to check the job status, once the job is done, it should be out of queue, takes ~3 mins
6. Once that's, check your folder for a **.out** file.
7. Read the output using `cat <OUTPUT_FILE_NAME>.out`

16. Moving files from your Docker Container to your AWS Cluster

1. Here's an example command to move multiple files from your Docker container/working folder into your the cluster:
`scp -i <KEY_NAME>.pem fit3143/task1.c fit3143/job.sh ec2-user@<publicIpAddress>:~/`
2. Or you can move the whole folder
`scp -i <KEY_NAME>.pem -r parallel-work ec2-user@<publicIpAddress>:~/`
3. You can get publicIpAddress from `pcluster describe-cluster --cluster-name mycluster --region ap-southeast-2`